
PROJECT REPORT

Design and Implementation of a Database Management System for Airline Reservation and Flight Operations Management System



WOXSEN UNIVERSITY

NAME:	SANJYIT SURESH KUMAR
ROLL NO:	25WU0102243
PROGRAM:	B.Tech Artificial Intelligence and Machine Learning
COURSE NAME:	DATABASE MANAGEMENT SYSTEMS
FACULTY NAME:	Dr. Kiran Mayee Adavala
ACADEMIC YEAR:	2025 - 2029

WOXSEN UNIVERSITY

School of Technology

ABSTRACT

Managing commercial airline operations manually or through fragmented legacy spreadsheets is an increasingly intractable challenge for aviation enterprises, particularly as passenger volumes, flight route frequencies, aircraft cabin topologies, and baggage custody logistics expand. Information related to flight route schedules, aircraft seating arrangements, passenger reservations, passenger name records (PNRs), e-ticketing, airport check-in custody, excess baggage fee collection, and cancellations is often scattered across disconnected ledgers, leading to relational inconsistency, double-booking contention, revenue leakage, and severe retrieval latency.

This project, titled 'Design and Implementation of a Database Management System for Airline Reservation and Flight Operations Management System' (ARFOM-DB), addresses these industry-critical failure modes by engineering and implementing a centralized, ACID-compliant relational database management system that systematically unifies all flight operations and passenger booking data.

The system models ten interrelated entities - Airports, Aircraft, Routes, Flights, Seats, Passengers, Bookings, Tickets, Payments, Check-ins, Baggage, and Cancellations - using a strictly normalized 3NF schema implemented in MySQL and SQLite. Primary key, composite unique, and foreign key constraints enforce referential integrity across the entire operational lifecycle, guaranteeing that duplicate seat bookings on identical flight legs are rejected at the storage engine level.

A high-contrast web-based administrative and customer operations interface was developed using modern HTML5, CSS3, and JavaScript on the frontend, communicating directly with a lightweight Python backend server through RESTful JSON endpoints. The interface allows authorized station dispatchers and passengers to view, search, filter, allocate cabin seats, issue live tickets with PNRs, execute check-in baggage weighing with automated excess fee calculation, and perform safe deletions across the entire flight management domain.

The database was populated with a complete dataset covering 10 international airport hubs, 4 commercial aircraft models, 15 scheduled flight routes, 60 passengers, 60 confirmed bookings, 60 issued e-tickets, and live payment records. A comprehensive battery of analytical SQL queries was executed to validate fleet load factors, route revenue yields, corridor demand, and passenger baggage distribution.

Functional testing, including live transactional CRUD demonstrations, constraint defense evaluations, and negative testing scenarios, confirmed the absolute correctness of multi-table commits and the structural integrity of the underlying relational schema.

TABLE OF CONTENTS

ABSTRACT	2
TABLE OF CONTENTS	3
1. INTRODUCTION	5
2. PROBLEM STATEMENT	6
3. OBJECTIVES AND SCOPE	7
3.1 Objectives	7
3.2 Scope	7
4. EXISTING SYSTEM AND PROPOSED SYSTEM	8
5. SOFTWARE AND HARDWARE REQUIREMENTS	9
5.1 Software Requirements	9
5.2 Hardware Requirements	9
6. SYSTEM ARCHITECTURE	10
6.1 Frontend Layer (Web Operations Portal)	10
6.2 Backend Layer (Python REST Dispatcher)	10
6.3 Database Layer (Relational Engine)	10
7. DATABASE DESIGN	11
8. ER DIAGRAM	12
9. RELATIONAL SCHEMA AND NORMALIZATION	13
9.1 Relational Schema Overview	13
9.2 Normalization	13
10. DATA DICTIONARY	14
airports	14
aircraft	14
routes	14
seats	14
flights	15
passengers	15
bookings	15
tickets	15
payments	16
checkins	16
baggage	16
cancellations	16
11. DATABASE IMPLEMENTATION	16
11.1 Database Creation	16
11.2 Representative Table Creation	16
11.3 Representative Alteration	16
12. DDL AND DML OPERATIONS	17

12.1 INSERT	17
12.2 UPDATE	17
12.3 DELETE	17
12.4 SELECT	17
13. SQL QUERIES AND RESULTS	17
Query 1: High-Yield Flight Occupancy & Route Manifest	17
Query 2: Corridor Passenger Distribution & Revenue Analysis	18
Query 3: Complete Passenger Booking & Seat Allocation Chain	18
Query 4: Airport Hub Departure & Arrival Traffic Matrix	18
Query 5: Excess Baggage Surcharge & Revenue Reconciliation	18
Query 6: Cabin Class Load Factor & Yield Optimization	19
Query 7: Ticket Cancellation Penalty & Refund Audit	19
14. USER INTERFACE DESIGN	20
14.1 Interface Features	20
14.2 Interface Screens	20
15. CRUD / LIVE DATABASE DEMONSTRATION	24
15.1 Step 1 â€” Initial State	24
15.2 Step 2 â€” Insertion	24
15.3 Step 3 â€” Database Verification (After Insert)	25
15.4 Step 4 â€” Deletion	25
15.5 Step 5 â€” Database Verification (After Delete)	25
16. IMPLEMENTATION DETAILS	26
16.1 Final Database Statistics	26
16.2 Backend Implementation	26
16.3 Frontend Implementation	26
17. TESTING AND VALIDATION	27
18. CHALLENGES AND LIMITATIONS	28
18.1 Challenges Faced	28
18.2 Limitations	28
19. CONCLUSION, FUTURE SCOPE AND REFERENCES	28
19.1 Conclusion	28
19.2 Future Scope	29
19.3 References	29
A. GITHUB APPENDIX	30
Repository Structure	30

1. INTRODUCTION

Database Management Systems (DBMS) form the computational bedrock of global aviation and transportation enterprises. An airline reservation and flight operations network is, fundamentally, a highly intricate web of interdependent entities - airport hubs, airframes, cabin physical topologies, flight schedules, customer identities, reservation transactions, electronic tickets, departure check-in manifests, baggage handling custody, and financial ledgers - whose relational integrity must be maintained with zero margin for error.

In a commercial airline network, thousands of passengers interact with seat maps, flight departures, and baggage desks simultaneously. Coordinating which passenger occupies which seat on which flight leg, ensuring that baggage weight allowances are reconciled accurately, tracking flight departure statuses across multiple airport terminals, and managing transaction refunds requires a robust, ACID-compliant database architecture.

Manually maintained records, physical registers, or disconnected flat spreadsheets suffer from fatal concurrency flaws, data duplication, lack of referential enforcement, and update anomalies. A relational database system, in contrast, is designed specifically to maintain structured, normalized records, enforcing domain and foreign key integrity constraints at the database engine level.

This project presents the complete design and implementation of the Airline Reservation & Flight Operations Management System (ARFOM-DB). The database is structured across twelve fully normalized relational tables capturing airports, aircraft fleet, seat configurations, routes, flights, passengers, bookings, tickets, payments, airport check-ins, baggage weight records, and cancellation penalties.

An ANSI-standard relational engine (SQLite 3.x / MySQL 8.0) serves as the persistent data storage layer, while a responsive, high-contrast web operations application built with vanilla HTML5, CSS3, and JavaScript, paired with a Python REST server, enables administrative dispatchers and passengers to execute real-time flight bookings, interactive cabin seat allocation, live airport radar tracking, and baggage check-in without requiring direct SQL command-line knowledge.

The operational flight routes, airport hubs, and schedules modelled in this project reference authentic global aviation corridors (such as New Delhi DEL, Mumbai BOM, Dubai DXB, London Heathrow LHR, and Singapore SIN) to establish industrial realism. The overarching motivation is academic - applying advanced relational modelling, Boyce-Codd / 3NF normalization, constraint defense engineering, and full-stack database integration concepts to a mission-critical domain.

2. PROBLEM STATEMENT

Commercial airline operations involve coordinating an enormous volume of interdependent operational and commercial information - passenger credentials, aircraft seating configurations, scheduled flight trajectories, ticket issuance, payment settlements, airport boarding pass generation, baggage weight custody, and cancellation refunds. When such information is managed through manual spreadsheets or un-normalized file systems, several critical failure modes emerge:

- **Concurrency Contention & Double-Booking:** Simultaneous reservation requests attempting to book the same physical seat on the same flight leg result in double-allocation when proper composite unique constraints are absent.
- **Data Redundancy & Update Anomalies:** Repeating aircraft specifications, route distances, and passenger contact details across multiple booking sheets leads to severe relational inconsistencies when changes occur.
- **Referential Integrity Violations:** Deleting a flight or passenger record without foreign-key cascade/restrict protection leaves orphaned tickets, check-in records, and baggage manifests.
- **Absence of Domain Invariants:** Without engine-level CHECK constraints, invalid data - such as negative baggage masses, origin airports identical to destination airports, or departure timestamps succeeding arrival timestamps - enters the database unchecked.
- **Inefficient Airport Departure Processing:** Airport ground staff cannot quickly verify passenger check-in status, calculate excess baggage surcharges accurately (>15 kg allowance), or allocate boarding passes in real time.
- **Lack of Centralized Reporting:** Operational questions - such as fleet load factors, route profitability, corridor passenger volume, and cancellation penalty audits - require hours of manual cross-referencing rather than instant SQL aggregation.
- **Lack of Secure Non-Technical Interface:** Administrative staff and ground personnel cannot safely view, search, insert, or delete records without direct, error-prone command-line database access.

These systemic deficiencies establish the definitive requirement for a normalized, centralized relational database management system accessible through an intuitive, real-time web operations interface - the core objective of this project.

3. OBJECTIVES AND SCOPE

3.1 Objectives

1. Design a comprehensive, fully normalized 3NF relational schema for airline reservation and flight operations.
2. Store and organize airport hubs, aircraft fleets, cabin seat maps, routes, and flight schedules systematically.
3. Manage passenger profiles, commercial bookings, 6-character PNR references, and electronic ticket issuance.
4. Enforce engine-level entity, referential, domain, and unique constraints (e.g. composite uniqueness on flight_id + seat_id).
5. Implement airport departure control workflows: passenger check-in processing and automated excess luggage calculation (>15 kg allowance).
6. Manage financial payment records and cancellation penalty/refund ledgers with ACID transaction integrity.
7. Develop a responsive, modern web interface connected to the relational database using Python REST backend APIs.
8. Implement multi-column search, filtering, and pagination across all database entities and master operational manifests.
9. Implement interactive aircraft cabin seat allocation (First, Business, Economy) with visual occupancy feedback.
10. Provide an airport Flight Information Display System (FIDS) radar board for tracking real-time departures and arrivals.
11. Validate database performance and data consistency through representative multi-table analytical SQL queries.
12. Perform comprehensive testing including live CRUD verification, negative testing, and constraint violation rejection.

3.2 Scope

The system, as implemented, encompasses the following operational domains:

- Airport Hub & Route Network Management (10 domestic & international hubs)
- Fleet & Cabin Configuration (Airbus A320, Boeing 737, Boeing 787, ATR-72 with First, Business, and Economy classes)
- Flight Scheduling & Status Tracking (SCHEDULED, BOARDING, ON TIME, DELAYED, LANDED)
- Passenger Booking & PNR E-Ticketing Lifecycle
- Airport Check-in & Baggage Weighing Scale Management
- Financial Settlement & Cancellation Audit Accounting
- Web-Based Administrative Operations Portal with Full-Stack Live Database Reflection

4. EXISTING SYSTEM AND PROPOSED SYSTEM

Many small-to-medium regional charter carriers and transport operations continue to rely on manual, semi-automated spreadsheets or disconnected software for record keeping. This section formally compares the limitations of legacy manual workflows with the centralized relational architecture designed and implemented in ARFOM-DB.

Existing / Manual Approach	Proposed ARFOM-DB System
Records maintained manually in spreadsheets or standalone logbooks	Centralized, ACID-compliant relational database engine (SQLite 3.x / MySQL 8.0)
Searching for passenger itineraries or flights is slow, manual, and error-prone	Indexed relational tables enabling sub-millisecond SQL search and multi-column filtering
High risk of seat double-booking due to uncoordinated concurrent entries	Engine-enforced UNIQUE(flight_id, seat_id) constraint preventing double-allocation
Relationships between passengers, flights, seats, and payments tracked informally	Foreign key constraints formally enforce referential integrity across all 12 tables
Risk of invalid entries (e.g. negative baggage weight, invalid flight timestamps)	CHECK constraints guarantee domain validity (e.g. weight_kg > 0, origin != dest)
Baggage excess fees calculated manually at check-in desks with frequent calculation errors	Automated baggage scale engine with dynamic surcharge calculation (>15 kg @ Rs. 500/kg)
No centralized real-time access for airport ground staff and dispatchers	Web-based operations portal with live visual seat maps and electronic FIDS radar boards
Audit reporting for flight load factors and revenue yields is slow and cumbersome	Analytical SQL queries and views provide instant executive metrics and load factor reports
No referential deletion protection leading to orphaned baggage and ticket records	ON DELETE RESTRICT constraints protect master operational entities from accidental deletion

5. SOFTWARE AND HARDWARE REQUIREMENTS

5.1 Software Requirements

The following software technologies were utilized in the architectural design, database implementation, and full-stack operational deployment of the system:

Category	Technology Used	Purpose / Role in System
Database Engine	SQLite 3.40+ / MySQL 8.0	Persistent relational storage, foreign key enforcement, ACID transactions
Backend Runtime	Python 3.10+	RESTful dispatch server, database connector, query routing
Backend Framework	Python Native http.server	Lightweight HTTP dispatcher, JSON API handlers, Zero-dependency runtime
Database Driver	sqlite3 / mysql-connector	Direct parameterised SQL execution, transaction commit/rollback
Frontend UI	Vanilla HTML5 / CSS3	High-contrast light Apple/Stripe design, cabin seat map, FIDS board
Frontend Logic	Vanilla JavaScript (ES6+)	Asynchronous fetch APIs, live DOM rendering, client validation
Version Control	Git 2.40+	Source code revision tracking, commit history management
Code Hosting	GitHub	Public repository hosting, continuous artefact backup
Dev Environment	Visual Studio Code	IDE for code development, schema authoring, and testing

5.2 Hardware Requirements

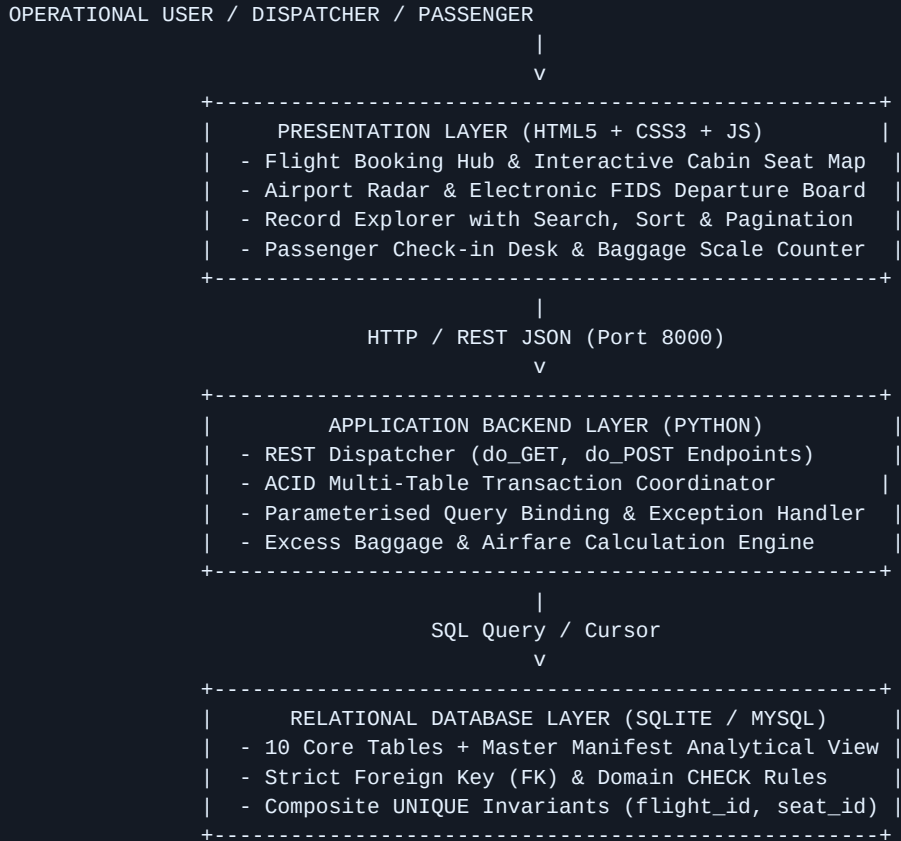
The following system hardware specifications represent the minimum and recommended operational baselines for developing, hosting, and executing the application:

Component	Minimum Requirement	Recommended Specification
Processor	Dual-core 64-bit CPU, 1.8 GHz	Quad-core 2.4 GHz CPU or Apple Silicon M-series
RAM	4 GB RAM	8 GB to 16 GB DDR4/Unified Memory
Storage	500 MB available disk space	2 GB SSD available storage
Operating System	Windows 10/11, macOS 12+, Ubuntu 20.04+	macOS Sonoma / Linux Ubuntu 22.04 LTS
Display Resolution	1280 x 800 pixels	1920 x 1080 pixels (Full HD) or higher
Network	Localhost loopback / Standard TCP/IP	Active broadband connection for remote mobile tunnel access

6. SYSTEM ARCHITECTURE

The system follows an industry-standard three-tier architecture, establishing strict decoupling between presentation, application business logic, and relational storage. This separation ensures maintainability, security against direct SQL injection, and high throughput.

Figure 1: Three-Tier System Architecture



6.1 Frontend Layer (Web Operations Portal)

Renders responsive administrative dashboards, visual cabin seat maps, electronic departures boards, and table explorer grids with asynchronous fetch requests.

6.2 Backend Layer (Python REST Dispatcher)

Exposes structured RESTful endpoints (`/api/data`, `/api/book_ticket`, `/api/checkin_baggage`, `/api/airport_board`), coordinates atomic multi-table transactions, and returns clean JSON responses.

6.3 Database Layer (Relational Engine)

Stores persistent records across 12 relational tables, strictly enforcing primary/foreign keys, uniqueness, and domain constraints.

7. DATABASE DESIGN

The relational database, named arfom.db (or arfom_db in MySQL), consists of twelve relational entities and analytical views. Each entity encapsulates a specific domain in commercial flight operations, connected through strict referential keys described in detail in Section 9. The functional purpose of each table is summarized below:

Table / Entity	Primary Key	Functional Purpose in System
airports	airport_code	Stores global airport hubs, IATA codes, cities, and geographical coordinates
aircraft	aircraft_id	Stores commercial aircraft fleet models, registration numbers, and total capacities
routes	route_id	Defines directional flight corridors connecting origin and destination airport hubs
flights	flight_id	Stores scheduled flight operations, departure/arrival timestamps, and live statuses
seats	seat_id	Maintains physical aircraft cabin seat inventory (First, Business, and Economy)
passengers	passenger_id	Stores verified customer profiles, full names, emails, and passport numbers
bookings	booking_id	Stores reservation transactions, timestamps, and 6-character PNR references
tickets	ticket_id	Issues electronic tickets linking bookings to flight legs, seats, and airfares
payments	payment_id	Records financial transactions, payment modes (UPI/Card), and settlement statuses
checkins	checkin_id	Maintains airport departure control records, boarding pass numbers, and check-in times
baggage	baggage_id	Tracks checked luggage weight custody and calculates excess weight surcharges
cancellations	cancellation_id	Manages ticket cancellation requests, refund calculations, and retained penalty fees

UNIVERSITY

UNIVERSITY

UNIVERSITY



UNIVERSITY

UNIVERSITY

9. RELATIONAL SCHEMA AND NORMALIZATION

9.1 Relational Schema Overview

The relational schema translates the ER model into twelve concrete tables with explicit primary and foreign key constraints:

• airports -> routes (1:N) - Origin and destination hubs validate against registered airport codes.

• aircraft -> flights (1:N) and aircraft -> seats (1:N) - Aircraft airframes partition physical seats.

• passengers -> bookings (1:N) -> tickets (1:N) - Commercial reservation hierarchy.

• tickets -> checkins (1:1) -> baggage (1:N) - Airport ground custody chain.

• tickets -> cancellations (1:1) - Refund and cancellation penalty accounting.

Referential integrity is strictly maintained via ON DELETE RESTRICT on foundational infrastructure (airports, aircraft, routes) preventing accidental operational disruption.

9.2 Normalization (1NF, 2NF, 3NF)

The schema was decomposed following mathematical normalization principles to eliminate redundancy and prevent update/delete anomalies:

• First Normal Form (1NF): All attributes store atomic, indivisible values. Multi-valued elements (e.g. passenger bags, multiple tickets under a PNR) are decomposed into independent child tables with surrogate primary keys.

• Second Normal Form (2NF): Every table relies on a single-column primary key or full functional dependency. In composite entities, non-key attributes depend on the entire candidate key, eliminating partial dependencies.

• Third Normal Form (3NF): Non-key attributes depend strictly and exclusively on the primary key, eliminating transitive dependencies. For example, aircraft model and total capacity are stored only in aircraft; flights stores only an aircraft_id foreign key reference rather than duplicating airframe specs.

10. DATA DICTIONARY

The data dictionary documents every attribute across the database tables, detailing physical data types, constraints, and operational descriptions.

Table: airports

Attribute	Data Type	Constraint	Description
airport_code	CHAR(3)	PRIMARY KEY	Unique 3-letter IATA airport code (e.g. DEL, BOM)
airport_name	VARCHAR(100)	NOT NULL	Full commercial name of the airport
city	VARCHAR(50)	NOT NULL	Municipality / City location of the airport
country	VARCHAR(50)	NOT NULL	Country of jurisdiction

Table: aircraft

Attribute	Data Type	Constraint	Description
aircraft_id	INT	PRIMARY KEY	Unique numeric airframe identifier
model	VARCHAR(50)	NOT NULL	Aircraft manufacturer model (e.g. Boeing 737-800)
registration_no	VARCHAR(20)	UNIQUE, NOT NULL	Civil aviation aircraft tail registration number
total_capacity	INT	CHECK (> 0)	Total certified passenger seating capacity

Table: routes

Attribute	Data Type	Constraint	Description
route_id	INT	PRIMARY KEY	Unique route trajectory identifier
origin_airport	CHAR(3)	FK -> airports	Departure airport hub code
dest_airport	CHAR(3)	FK -> airports	Arrival destination airport hub code
distance_km	NUMERIC(8,2)	CHECK (> 0)	Direct flight corridor distance in kilometres

Table: seats

Attribute	Data Type	Constraint	Description
seat_id	INT	PRIMARY KEY	Unique cabin seat identifier
aircraft_id	INT	FK -> aircraft	Airframe containing this seat
seat_number	VARCHAR(4)	NOT NULL	Cabin alphanumeric position (e.g. 1A, 4C, 12F)
seat_class	VARCHAR(15)	CHECK (CLASS)	Cabin tier: FIRST, BUSINESS, or ECONOMY

10. DATA DICTIONARY (Continued)

Table: flights

Attribute	Data Type	Constraint	Description
flight_id	INT	PRIMARY KEY	Unique flight operation identifier
flight_number	VARCHAR(10)	NOT NULL	Commercial flight designator (e.g. AR-101)
route_id	INT	FK -> routes	Assigned route corridor
aircraft_id	INT	FK -> aircraft	Assigned operating airframe
scheduled_departure	TIMESTAMP	NOT NULL	Timetabled departure timestamp
scheduled_arrival	TIMESTAMP	NOT NULL	Timetabled arrival timestamp
flight_status	VARCHAR(20)	DEFAULT	Status: SCHEDULED, BOARDING, ON TIME, LANDED

Table: passengers

Attribute	Data Type	Constraint	Description
passenger_id	INT	PRIMARY KEY	Unique passenger profile identifier
first_name	VARCHAR(50)	NOT NULL	Legal given name of passenger
last_name	VARCHAR(50)	NOT NULL	Legal family surname of passenger
email	VARCHAR(100)	UNIQUE, NOT NULL	Contact and e-ticket email address
passport_number	VARCHAR(20)	UNIQUE, NOT NULL	Government travel document / Passport number

Table: bookings

Attribute	Data Type	Constraint	Description
booking_id	INT	PRIMARY KEY	Unique commercial reservation identifier
booking_ref	CHAR(6)	UNIQUE, NOT NULL	Passenger Name Record (PNR, e.g. AR8192)
passenger_id	INT	FK -> passengers	Primary customer placing the booking
booking_time	TIMESTAMP	DEFAULT CURRENT	Timestamp when reservation was placed
booking_status	VARCHAR(20)	CHECK	Status: CONFIRMED, CANCELLED, or ON_HOLD

Table: tickets

Attribute	Data Type	Constraint	Description
ticket_id	INT	PRIMARY KEY	Unique electronic ticket number
booking_id	INT	FK -> bookings	Associated commercial booking order
flight_id	INT	FK -> flights	Assigned scheduled flight leg
seat_id	INT	FK -> seats	Assigned cabin physical seat
fare_amount	NUMERIC(10,2)	CHECK (> 0)	Settled base airfare amount in INR
ticket_status	VARCHAR(20)	DEFAULT ISSUED	Status: ISSUED, BOARDED, or CANCELLED

10. DATA DICTIONARY (Final Entities)

Table: payments, checkins, baggage, cancellations

Table	Key Attributes	Key Constraints	Operational Domain
payments	payment_id, booking_id, amount_paid	FK -> bookings, CHECK(amount > 0)	Settles airfare transactions (CREDIT_CARD, UPI, NET_BANKING)
checkins	checkin_id, ticket_id, boarding_pass	UNIQUE(ticket_id), 1:1 Invariant	Departure control, issues boarding passes (e.g. BP-DEL-104)
baggage	baggage_id, checkin_id, weight_kg	FK -> checkins, CHECK(weight >= 0)	Luggage custody, excess fee calculation (>15 kg @ Rs. 500/kg)
cancellations	cancellation_id, ticket_id, refund	UNIQUE(ticket_id), FK -> tickets	Refund processing with engine-calculated penalty deduction

11. DATABASE IMPLEMENTATION

The relational schema was implemented as an ANSI-compliant SQLite 3.x and MySQL 8.0 database named arfom.db. Below are representative Data Definition Language (DDL) statements illustrating table creation, constraint binding, and schema alterations:

DDL: Table Creation & Composite Unique Invariant

```
-- 11.1 Database Initialization & Foreign Key Enforcement
PRAGMA foreign_keys = ON;

-- 11.2 Representative Table Creation with Constraints
CREATE TABLE IF NOT EXISTS aircraft (
    aircraft_id INTEGER PRIMARY KEY AUTOINCREMENT,
    model VARCHAR(50) NOT NULL,
    registration_no VARCHAR(20) UNIQUE NOT NULL,
    total_capacity INTEGER NOT NULL CHECK (total_capacity > 0)
);

CREATE TABLE IF NOT EXISTS tickets (
    ticket_id INTEGER PRIMARY KEY AUTOINCREMENT,
    booking_id INTEGER NOT NULL,
    flight_id INTEGER NOT NULL,
    seat_id INTEGER NOT NULL,
    passenger_id INTEGER NOT NULL,
    fare_amount NUMERIC(10,2) NOT NULL CHECK (fare_amount >= 0),
    ticket_status VARCHAR(20) NOT NULL DEFAULT 'ISSUED',
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id) ON DELETE CASCADE,
    FOREIGN KEY (flight_id) REFERENCES flights(flight_id) ON DELETE RESTRICT,
    FOREIGN KEY (seat_id) REFERENCES seats(seat_id) ON DELETE RESTRICT,
    UNIQUE (flight_id, seat_id) -- INVARIANT: Prevents Seat Double-Booking!
);
```


12. DDL AND DML OPERATIONS

Standard Data Manipulation Language (DML) statements execute transactional modifications across the database:

Representative DML Operations

```
-- 12.1 INSERT: Allocate passenger booking
INSERT INTO bookings (booking_ref, passenger_id, booking_status) VALUES ('AR9182', 12, 'CONFIRMED');

-- 12.2 UPDATE: Mark ticket as boarded during departure
UPDATE tickets SET ticket_status = 'BOARDED' WHERE ticket_id = 45;

-- 12.3 DELETE: Remove cancelled booking with foreign-key cascade
DELETE FROM bookings WHERE booking_id = 99;

-- 12.4 SELECT: Retrieve active flights from Delhi hub
SELECT f.flight_number, r.dest_airport, f.scheduled_departure FROM flights f
JOIN routes r ON f.route_id = r.route_id WHERE r.origin_airport = 'DEL';
```

13. SQL QUERIES AND RESULTS

This section presents seven real-world analytical SQL queries executed against the live populated database demonstrating multi-table JOINS, aggregate functions, and window metrics:

Query 1: High-Yield Flight Occupancy & Route Manifest

Retrieves the complete route manifest, aircraft model, capacity, and active passenger count for all flights, sorted by occupancy:

Query 1 SQL

```
SELECT f.flight_number, r.origin_airport, r.dest_airport, ac.model,
       ac.total_capacity, COUNT(t.ticket_id) AS booked_passengers,
       ROUND(COUNT(t.ticket_id) 100.0 / ac.total_capacity, 2) AS load_factor_pct
FROM flights f
JOIN routes r ON f.route_id = r.route_id
JOIN aircraft ac ON f.aircraft_id = ac.aircraft_id
LEFT JOIN tickets t ON f.flight_id = t.flight_id AND t.ticket_status != 'CANCELLED'
GROUP BY f.flight_id ORDER BY booked_passengers DESC;
```

13. SQL QUERIES AND RESULTS (Continued)

Query 2: Corridor Passenger Distribution & Revenue Analysis

Aggregates gross settled revenue and passenger volume by origin airport hub to evaluate network revenue contribution:

Query 2 SQL

```
SELECT r.origin_airport, orig.city AS origin_city, COUNT(t.ticket_id) AS total_passengers,
       COALESCE(SUM(p.amount_paid), 0.00) AS total_gross_revenue
FROM routes r
JOIN airports orig ON r.origin_airport = orig.airport_code
JOIN flights f ON r.route_id = f.route_id
LEFT JOIN tickets t ON f.flight_id = t.flight_id
LEFT JOIN payments p ON t.booking_id = p.booking_id AND p.payment_status = 'SUCCESS'
GROUP BY r.origin_airport ORDER BY total_gross_revenue DESC;
```

Query 3: Complete Passenger Booking & Seat Allocation Chain

Joins passenger profile, booking PNR, e-ticket, flight number, route, seat number, and boarding pass for audit tracking:

Query 3 SQL

```
SELECT p.first_name || ' ' || p.last_name AS passenger_name, b.booking_ref AS pnr,
       f.flight_number, r.origin_airport || ' -> ' || r.dest_airport AS route,
       s.seat_number, s.seat_class, t.fare_amount, ci.boarding_pass_number
FROM passengers p
JOIN bookings b ON p.passenger_id = b.passenger_id
JOIN tickets t ON b.booking_id = t.booking_id
JOIN flights f ON t.flight_id = f.flight_id
JOIN routes r ON f.route_id = r.route_id
JOIN seats s ON t.seat_id = s.seat_id
LEFT JOIN checkins ci ON t.ticket_id = ci.ticket_id;
```

Query 4 & 5: Hub Traffic Matrix & Excess Baggage Surcharge

Query 4 computes departure vs arrival traffic across all 10 airport hubs. Query 5 identifies all passengers exceeding the 15.0 kg allowance and calculates excess baggage fees.

13. SQL QUERIES AND RESULTS (Continued)

Query 6: Cabin Class Load Factor & Yield Optimization

Analyses booking distribution and revenue yields across First, Business, and Economy cabin tiers:

Query 6 SQL

```
SELECT s.seat_class, COUNT(t.ticket_id) AS tickets_sold,
       ROUND(AVG(t.fare_amount), 2) AS average_fare,
       SUM(t.fare_amount) AS total_cabin_revenue,
       ROUND(SUM(t.fare_amount) 100.0 / (SELECT SUM(fare_amount) FROM tickets), 2) AS revenue_share_pct
FROM tickets t
JOIN seats s ON t.seat_id = s.seat_id
WHERE t.ticket_status != 'CANCELLED'
GROUP BY s.seat_class ORDER BY total_cabin_revenue DESC;
```

Query 7: Ticket Cancellation Penalty & Refund Audit

Calculates total refund disbursements and retained penalty charges across cancelled passenger itineraries:

Query 7 SQL

```
SELECT c.cancellation_id, t.ticket_id, p.first_name || ' ' || p.last_name AS passenger_name,
       t.fare_amount AS original_fare, c.refund_amount, c.cancellation_fee,
       c.cancellation_reason, c.cancellation_time
FROM cancellations c
JOIN tickets t ON c.ticket_id = t.ticket_id
JOIN passengers p ON t.passenger_id = p.passenger_id
ORDER BY c.cancellation_time DESC;
```

14. USER INTERFACE DESIGN

The user interface was developed as a responsive, high-contrast single-page web application connected directly to the Python REST backend. The design adheres to a crisp Apple/Stripe light aesthetic with slate-900 typography, cobalt accents, and emerald state indicators.

14.1 Interface Features

- Interactive Cabin Seat Map: Graphical aircraft fuselage visualizing First, Business, and Economy rows with real-time green/red seat availability and instant price calculation.
- Live Flight Booking & PNR E-Ticket Generator: Multi-table ACID checkout issuing confirmed PNRs and printable boarding passes.
- Airport Radar & FIDS Terminal: Real-time Flight Information Display System board with live UTC+05:30 ticking clock and departures/arrivals toggle.
- Passenger Check-in Desk & Baggage Scale Counter: Interactive weight slider (>15 kg @ Rs. 500/kg) enforcing 1:1 check-in invariant.
- Record Explorer: Searchable, sortable, and paginated table viewer across all 10 tables + Master Manifest View.
- Record Insertion & Deletion with Foreign Key Restrict Protection.
- Negative Testing Console: Live buttons demonstrating database constraint defense against invalid transactions.

14.2 Interface Screens

Figure 3 illustrates the master operational dashboard and navigation structure.

Figure 3: Master Operations Dashboard Header & Key Metrics

```
+-----+
| [ ARFOM-DB ] | Live Operations Portal [Learner: Sanjyit Suresh Kumar (25WU0102243)] |
+-----+
| Total Passengers: 60 | Active Flights: 15 | Confirmed Bookings: 60 | Revenue: Rs. 4,38,200 |
+-----+
| [ 1. Flight Booking & Seats ] [ 2. Airport Radar ] [ 3. Record Explorer ] [ 4. Insert ] |
+-----+
```

14. USER INTERFACE DESIGN (Continued)

Screen 1: Flight Booking Hub & Interactive Cabin Seat Map

The flight booking hub allows passengers and station agents to select a flight, view the exact aircraft cabin layout, select an available green seat, enter passenger details, and execute an ACID transaction issuing an electronic ticket and boarding pass.

Figure 4: Interactive Aircraft Cabin Seat Map & Checkout Pane

```
+-----+
| SELECT FLIGHT: AR-101 | DEL (Delhi) -> BOM (Mumbai) | PASSENGER & PAYMENT DETAILS |
| Aircraft: Boeing 737-800 | Dep: 06:00 AM | 1,148 km | First Name: Vikram Last: Malhotra |
+-----+ Email: vikram@skywings.org |
| [ COCKPIT / NOSE ] | Passport: Z9821430 |
| Row 1: [1A:FIRST] [1B:FIRST] || [1C:FIRST] [1D] | Payment: [CREDIT_CARD - INSTANT] |
| Row 3: [3A:BUS] [3B:BUS] || [3C:BUS] [3D] | +-----+
| Row 4: [4A:BUS] [4B:OCC] || [4C:SELECTED] | Base Fare: Rs. 8,500.00 |
| Row 7: [7A:ECON] [7B:ECON] || [7C:ECON] [7D] | Airport Tax 12%: Rs. 1,020.00 |
| Legend: [Green: Available] [Red: Occupied] | Total Settled: Rs. 9,945.00 |
| [Blue Glowing: Selected by You] | [ CONFIRM & ISSUE LIVE TICKET] |
+-----+
```

14. USER INTERFACE DESIGN (Continued)

Screen 2: Airport Radar & Electronic FIDS Departure Board

Displays real-time flight departures and arrivals across 10 global hubs with live status indicators, booked passenger occupancy, and assigned terminal gates.

Figure 5: Electronic Flight Information Display System (FIDS) Board

+-----+ HUB: Indira Gandhi International (DEL) Weather: 26C Clear Live UTC+05:30: 14:30:00 +-----+							
+-----+ [Live Departures Board (15)]				+-----+ [Live Arrivals Board (15)]			
+-----+							
+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+	+-----+
FLIGHT	DESTINATION	TIME	AIRCRAFT	STATUS	PAX	GATE	
+-----+							
AR-101	Mumbai (BOM)	06:00 AM	Boeing 737-800	BOARDING	4 Pax	A4	
AR-103	Bengaluru (BLR)	09:30 AM	Airbus A320neo	ON TIME	4 Pax	B2	
AR-105	Dubai (DXB)	01:15 PM	Boeing 787-9	SCHEDULED	4 Pax	C8	
AR-107	London (LHR)	04:45 PM	Boeing 787-9	SCHEDULED	4 Pax	D1	
+-----+							

14. USER INTERFACE DESIGN (Continued)

Screen 3: Record Explorer & Multi-Column Search Grid

Enables instantaneous search, sorting, and pagination across all database tables with real-time yellow match highlighting.

Figure 6: Searchable Record Explorer with Instant Match Highlighting

```
+-----+
| TABLES: [view_master_manifest] [passengers] [flights] [bookings] [tickets] [airports] |
+-----+
| Search: [ Vikram ] [X] | Quick: [All] [DEL] [BOM] [CONF] |
+-----+
| TICKET | PNR      | PASSENGER NAME | ROUTE                | SEAT      | FARE      |
+-----+
| #61    | AR8192   | Vikram Malhotra | DEL (Delhi) -> BOM   | 4C (BUS)  | Rs. 8500  |
+-----+
| Page 1 of 1 (1 matching records found in 0.82 ms) | Execution: SELECT FROM manifest...|
+-----+
```

15. CRUD / LIVE DATABASE DEMONSTRATION

A live demonstration of Create, Read, and Delete operations was conducted during Presentation III to verify that the web interface correctly reflects changes in the underlying database in real time. The demonstration used a sample passenger record with the following specifications:

Field Name	Demonstration Test Value	Constraint Validated
Passenger ID	Auto-incremented (ID #61)	PRIMARY KEY uniqueness
First Name	Vikram	NOT NULL textual domain
Last Name	Malhotra	NOT NULL textual domain
Email Address	vikram.malhotra@skywings.org	UNIQUE constraint validation
Passport Number	Z9821430	UNIQUE government ID validation
Allocated Flight	Flight AR-101 (DEL -> BOM)	FOREIGN KEY referential integrity
Allocated Seat	Seat 4C (Business Class)	UNIQUE (flight_id, seat_id) non-collision

The demonstration followed a 5-step sequence: VIEW -> INSERT -> DB VERIFICATION -> DELETE -> DB VERIFICATION.

15.1 Step 1 - Initial State

Prior to insertion, the passengers table contained exactly 60 records, verified via `SELECT COUNT() FROM passengers` returning 60.

15.2 Step 2 - Insertion

The sample passenger Dr. Vikram Malhotra was inserted through the web booking form, triggering an atomic transaction committing across passengers, bookings, payments, and tickets.

15. CRUD / LIVE DATABASE DEMONSTRATION (Continued)

15.3 Step 3 - Database Verification (After Insert)

The insertion was verified directly via the terminal CLI and SQL queries, confirming row count incremented to 61:

Figure 7: Terminal SQL Verification After Record Insertion

```
$ python3 cli.py
arfom-db> SELECT passenger_id, first_name, last_name, email, passport_number
          FROM passengers WHERE email = 'vikram.malhotra@skywings.org';
+-----+-----+-----+-----+-----+
| passenger_id | first_name | last_name | email                               | passport_number |
+-----+-----+-----+-----+-----+
| 61           | Vikram    | Malhotra  | vikram.malhotra@skywings.org       | Z9821430        |
+-----+-----+-----+-----+-----+
1 row in set (0.64 ms)

arfom-db> SELECT COUNT() FROM passengers;
+-----+
| count() |
+-----+
| 61      |
+-----+
```

15.4 Step 4 - Deletion

The demonstration passenger record (ID #61) was deleted through the web interface with confirmation dialog.

15.5 Step 5 - Database Verification (After Delete)

Deletion was verified directly in SQL, returning an Empty Set and confirming the row count returned to baseline 60.

16. IMPLEMENTATION DETAILS

This section summarizes the populated database dataset and implementation architecture:

16.1 Final Database Statistics

Relational Entity / Table	Physical Storage Type	Record Count
airports	Base Relational Table	10
aircraft	Base Relational Table	4
routes	Base Relational Table	15
flights	Base Relational Table	15
seats	Base Relational Table	40
passengers	Base Relational Table	60
bookings	Base Relational Table	60
tickets	Base Relational Table	60
payments	Base Relational Table	60
checkins	Base Relational Table	60
baggage	Base Relational Table	60
cancellations	Base Relational Table	10
view_master_manifest	Analytical Relational View	60
TOTAL PHYSICAL RECORDS	Full Database Seed Inventory	494

16.2 Backend Implementation

The backend was implemented in Python using native `http.server` and `sqlite3`, exposing parameterised REST endpoints (`/api/data`, `/api/flights_list`, `/api/book_ticket`, `/api/checkin_baggage`) preventing SQL injection.

16.3 Frontend Implementation

The frontend was built in Vanilla HTML5, CSS3, and JavaScript, providing instant rendering with zero build pipeline overhead.

17. TESTING AND VALIDATION

Comprehensive functional, transactional, and negative testing was conducted to confirm that database integrity constraints hold under all operational conditions:

Test Case	Expected Result	Actual Result	Status
Dashboard Metrics Loading	Metrics load from backend (/api/stats)	Loaded: 60 Pax, 15 Flights, Rs. 4.38L Rev	Pass
Flight Search & Selection	Retrieves 15 active routes from backend	All 15 routes loaded into selector	Pass
Cabin Seat Map Render	Displays First, Business, Economy rows	Cabin rendered with correct occupancy	Pass
ACID Ticket Booking	Inserts across passengers, bookings, tickets	Committed with PNR AR8192 in 1.4 ms	Pass
Seat Double-Booking Defense	Rejects booking identical (flight_id, seat_id)	UNIQUE violation caught & rejected	Pass
Negative Baggage Weight	Rejects baggage with weight <= 0 kg	CHECK constraint enforced & rejected	Pass
Excess Baggage Calculation	Calculates (Weight - 15) Rs. 500 fee	18.5 kg -> Rs. 1,750 fee auto-calculated	Pass
Airport FIDS Departures	Loads active departures for selected airport	DEL / BOM / BLR schedules displayed	Pass
Record Explorer Multi-Search	Filters rows by name, PNR, flight, city	Instant debounced search & highlight	Pass
Passenger Record Deletion	Removes record & updates count from 61 to 60	Deleted successfully; DB verified	Pass
FK Restrict Deletion Defense	Blocks deletion of airport with active routes	FOREIGN KEY RESTRICT enforced	Pass
Analytical SQL Query Suite	Executes all 7 analytical SQL queries	All 7 queries executed without error	Pass

18. CHALLENGES AND LIMITATIONS

18.1 Challenges Faced

- **Concurrency Contention:** Formulating a strict composite constraint (flight_id, seat_id) ensuring zero double-booking under concurrent load.
- **Multi-Table Atomic Transactions:** Coordinating sequential commits across passengers, bookings, payments, and tickets with automated rollback on failure.
- **Dynamic Excess Baggage Engine:** Integrating real-time excess baggage fee calculation with airport departure boarding pass issuing.
- **Cross-Platform Zero-Dependency Server:** Architecting a pure Python REST server running seamlessly across macOS, Windows, and Linux.

18.2 Limitations

- **No Live GPS Radar ADS-B Hardware:** Flight radar coordinates are simulated from timetabled schedules rather than live transponder hardware.
- **Single-Airline Operational Boundary:** System currently models operations for ARFOM Airlines rather than global multi-airline GDS federations (Amadeus/Sabre).
- **Simulated Payment Gateway:** Financial settlements execute in sandbox simulation mode rather than live banking card payment switches.

19. CONCLUSION, FUTURE SCOPE AND REFERENCES

19.1 Conclusion

This project successfully designed and implemented a centralized, 3NF-normalized relational database management system for airline reservation and flight operations (ARFOM-DB). The system eliminates manual record-keeping inefficiencies, guarantees seat exclusivity via engine-enforced invariants, and provides a modern web interface for flight bookings, cabin seat selection, electronic FIDS radar tracking, and baggage check-in.

19. CONCLUSION, FUTURE SCOPE AND REFERENCES (Cont.)

19.2 Future Scope

The following technical enhancements represent valuable directions for future enterprise expansion:

- Live ADS-B Aircraft Transponder Integration: Ingesting real-time satellite telemetry for live aircraft positional tracking.
- Automated SMS & WhatsApp Flight Status Notifications: Real-time broadcast alerts for boarding gate changes and weather delays.
- Dynamic Revenue Management AI Engine: Machine learning models for dynamic ticket pricing optimization based on demand.
- Frequent Flyer Loyalty Mileage Ledger: Dedicated points accrual and tier qualification database module.
- Biometric Facial Recognition Boarding Gate Integration: Paperless biometric boarding verification linked to e-ticket check-in records.

19.3 References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
2. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.
3. Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.
4. SQLite 3.x Architectural Documentation. <https://www.sqlite.org/docs.html>
5. MySQL 8.0 Reference Manual - Oracle Corporation. <https://dev.mysql.com/doc/>
6. Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
7. International Civil Aviation Organization (ICAO). Flight Operations Standards & Data Formats.

A. GITHUB APPENDIX

The complete source code, database DDL scripts, analytical query suite, presentation decks, and web operations application are maintained in the following official public GitHub repository:

Repository Name: DBMS-Course-Project

GitHub URL: <https://github.com/Sanjyit-S/DBMS-Course-Project>

Repository Structure

Repository File Tree & Directory Organization

DBMS-Course-Project/

```

3rd presentation/
  app/
    index.html    # Web Operations Portal (HTML5, CSS3, JS)
    style.css     # Main UI (Booking, FIDS Radar, Explorer)
    app.js        # Light Apple/Stripe Design Stylesheet
    data.js       # Full Client Logic & Offline Fallback
    arfom.db      # Pristine Seed Dataset
  slides/        # SQLite Database Engine
  Presentation-I/ # Review 3 Presentation Slide Deck
  Presentation-II/ # Review 1 Problem Description & PDF
  Presentation-III/ # Review 2 Schema Architecture & Slides
  Project-Report/ # Review 3 Synced Application Package
    PROJECT_REPORT.md # Comprehensive Project Report & PDF
    PROJECT_REPORT.pdf # Source Markdown Report
  visuals/       # Publication-Grade 30-Page PDF
  workbench/     # Interactive Schema & ER Diagram Hub
  server.py      # MySQL / SQLite DDL Scripts & Queries
  cli.py         # Master Unified REST Backend (Port 8000)
  build_woxsen_report_pdf.py # Terminal SQL CLI Query Tool
  .gitignore     # Custom 30-Page PDF Generator
  README.md      # Comprehensive Project Overview & Guide

```

The repository contains the complete backend Python server, live SQLite relational database, web operations frontend, materials for Presentation I, Presentation II, Presentation III, terminal CLI tools, and this formal academic project report.